1 · The problem

- Dead Python code accumulates: unused imports, helpers, abandoned classes.
- Naive auto-delete is dangerous — false positives break production.
- Running untrusted repo tooling on a laptop = unbounded blast radius.
- Hackathon track Blast Radius Zero demands proven containment, not slides alone.

2 · Wake.ai — the cleanup crew for your code

- Clone → harden → scan (Vulture + ruff) → triage → prove with pytest → graveyard.
- Control plane never runs user code; keys stay off the sandbox host.
- Removals recorded in wake.md + wake.lock.json for exact resurrect.
- lablab rule: AI planner/triage/narrator stubbed until build week 3–8 Nov 2026.

3 · How it works

- Accept demo id / GitHub URL / zip (≤5MB, ≤300 files).
- Sandbox: runsc, cap-drop ALL, no-new-privileges, uid 65532, RO rootfs, tmpfs workspace.
- Network on for clone/install → then network none.
- Prove loop: baseline → batch remove → re-test → split on failure → restore culprit.
- Destroy container; 15-minute sweeper as safety net. Rate limit 3/IP/hr.

4 · Containment proof (runsc evidence)

- Runtime runsc · uid 65532 · docs/poisoned_carcass_evidence.json · summary_pass true.
- rm -rf /: Permission denied on RO root; python still alive; host untouched.
- Fork bomb: exit 128 / gVisor WaitPID EOF; sandbox stopped; host healthy.
- Infinite loop: exit 137, killed at ~60.03s (timeout -s KILL).
- Env/metadata/example.com: WAKE_SANDBOX=1, no host secrets, fetches fail (net none).
- Aftermath: 0 containers left on host.

5 · Self-healing prove loop

- Never delete without a green pytest after the change.
- Batch removals for speed; on failure binary-split to find the culprit.
- Culprit restored and marked alive; safe removals stay deleted.
- e2e on runsc: tidy-todo 9 removals / 0 apply errors; flask-blog 4 removals.

6 · Graveyard & resurrect

- wake.md — human-readable graveyard with hunks and resurrect commands.
- wake.lock.json — id, file, base SHA, hunk, sha256, optional depends_on.
- UI + API: resurrect one or all; scripts/wake_resurrect.py for CLI.
- Download cleaned tree as zip after a job.

7 · Architecture

- Two VMs: Control plane (API, SQLite, UI, LLM in build week) | Sandbox host (Docker/gVisor only).
- Hardened image pinned: python:3.12.7-slim-bookworm@sha256:60d9996b…
- Replay fixtures under replay/ when spend cap / rate limit / ?replay=1.
- Honest gap: VM iptables metadata firewall unverified until Vultr deploy.

8 · Value & roadmap

- Value: safer dead-code cleanup with auditable resurrect and proven blast-radius zero.
- Now: deterministic stubs, real runsc containment, replay mode, SPA UI.
- Build week (3–8 Nov 2026): Vultr Serverless Inference for triage/planner/narrator.
- Deploy: two Vultr VMs, mTLS to sandbox host, metadata firewall, public demo URL.
- Credits: Vulture, ruff, Knip (inspiration), gVisor. License: MIT.